

SYSTEM ARCHITECTURE

AI Agent Governance Platform

A reference document for the engineering team — architecture, data model, request flow, and design decisions with their rationale.

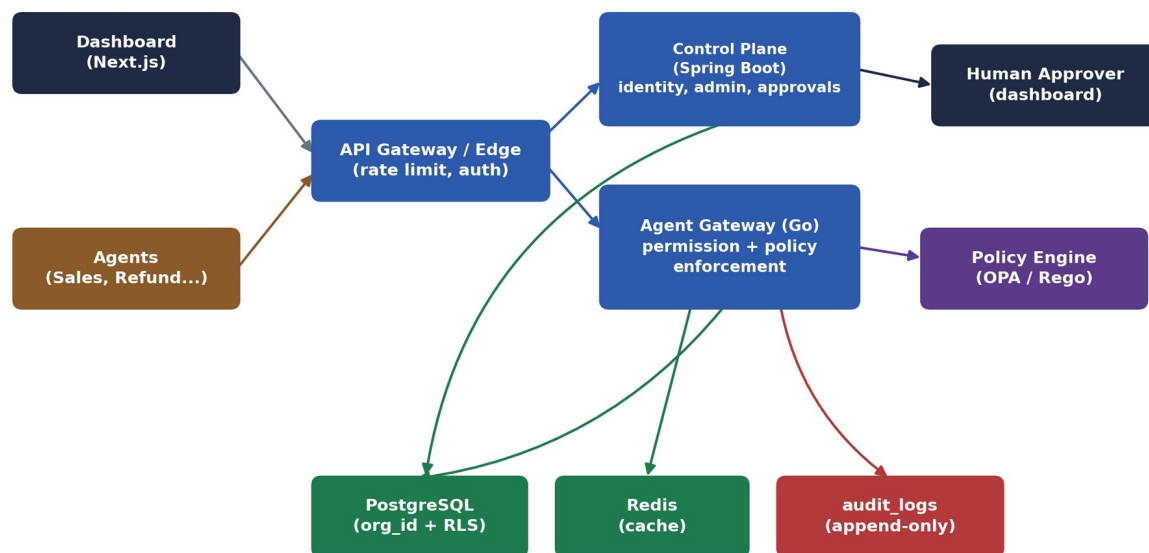
1. System Overview

The platform is a governance control plane that sits between AI agents and the tools they call. Every agent action — reading a CRM record, issuing a refund, pushing code — is required to pass through the Agent Gateway, which checks the agent's permissions, evaluates policy, and either executes the action automatically or routes it to a human approver. Every decision, allowed or blocked, is written to an immutable audit log.

Four properties drive every architectural decision in this document: the Gateway must be low-latency since it sits in the hot path of every action; permissions and policy must be enforceable and auditable, not just documented; approvals must involve a real human in the loop for high-risk actions; and the system must scale from one organization today to many organizations later without a rewrite.

2. Request Flow Diagram

AI Agent Governance Platform — Request Flow



Solid boxes = services • Arrows = request/data flow • All state lives in Postgres, keyed by org_id

Both the Dashboard and every Agent talk to the Control Plane and Gateway through a shared API Gateway/Edge layer, which handles authentication and per-tenant rate limiting. The Agent Gateway is the only component that talks to the Policy Engine directly, and it — not the Control Plane — is what actually executes or defers an action. The Control

Plane owns everything administrative: registering agents, managing permissions and policies, and surfacing the approval queue to human approvers.

3. Services

Service	Language	Responsibility
Gateway	Go	Latency-critical enforcement path: authenticates the agent, checks permissions, evaluates policy via OPA, executes or defers to approval, writes the audit event.
Control Plane	Java / Spring Boot	Agent registry, permission/policy administration, the human approval queue, and (later) SSO/SAML — everything that is not latency-sensitive.
Agent Runner	Python / FastAPI	Hosts the mock/demo agents (Sales Agent, Refund Agent) that call the Gateway exactly as a real agent integration would.
Frontend	Next.js / TypeScript	Admin dashboard: agent registry, approval queue, audit explorer.
Policy Engine	OPA / Rego	Evaluates governance policy (e.g. refund thresholds) against each action, called by the Gateway.

Why two backend languages: the Gateway is a narrow, latency-critical proxy — Go's low overhead and simple concurrency model fit that job well. Everything else in the Control Plane is admin CRUD, identity, and approval-workflow logic, where Spring Boot's mature security/OIDC/SAML tooling saves significant engineering time — and that code is not on the hot path, so the JVM's heavier footprint doesn't cost you anything there.

4. Request Flow — Two Worked Examples

4.1 Low-risk action (auto-approved)

1. Refund Agent calls the Gateway: action=refund, amount=750.
2. Gateway checks permissions for this agent/tool/action — allowed.
3. Gateway calls OPA with the refund policy input; amount < 1000 → decision=auto.
4. Gateway calls the (mocked, in POC) payment tool directly.
5. Gateway writes a tool_calls row and an audit_logs event; response returned to the agent synchronously.

4.2 High-risk action (human approval required)

1. Refund Agent calls the Gateway: action=refund, amount=25000.
2. Permission check passes; OPA returns decision=finance.
3. Gateway calls the Control Plane to create an approvals row (status=pending) instead of executing.
4. The approval appears in the dashboard's approval queue; a finance approver approves or rejects it.
5. On approval, the deferred action executes and is logged; on rejection, it's logged as blocked. Either way, an audit_logs event is written.

5. Data Model

Every table carries org_id from day one, even though the current build is single-organization. This means turning on Postgres Row-Level Security later — the mechanism that enforces tenant isolation — is a configuration change, not a

migration that touches every row and every query. See `db/migrations/V1__init_schema.sql` in the repo for the full, current schema.

Table	Key Columns	Purpose
organizations	id, name, created_at	Tenant boundary — every other table references this.
users	id, org_id, email, role	Human operators and approvers.
agents	id, org_id, name, status, owner_id	Registered agents.
agent_identities	id, org_id, agent_id, api_key_hash	Service-account style credential per agent.
permissions	id, org_id, agent_id, tool, action, allowed	Per-agent, per-tool, per-action grants.
policies	id, org_id, name, rule_json	Policy definitions (mirrored into OPA/Rego for evaluation).
approvals	id, org_id, agent_id, action, status, approver_id, created_at, resolved_at	Pending / approved / rejected human-approval requests.
executions	id, org_id, agent_id, status, started_at, completed_at	One row per agent run.
tool_calls	id, org_id, execution_id, tool, action, payload, result, latency_ms	Every gateway-mediated call to a downstream tool.
audit_logs	id, org_id, execution_id, event_type, detail_json, created_at	Append-only trail of every decision and action.

6. Key Design Decisions & Rationale

- Two backend languages, one clear boundary — Go owns the hot path (Gateway), Spring Boot owns admin/identity/approvals (Control Plane). Neither service calls into the other's database tables directly; they talk over HTTP.
- OPA/Rego for policy, not a custom rules engine — avoids building and maintaining a bespoke policy DSL, and gives per-tenant policy bundles a natural home later.
- `org_id` on every table from the first migration — the single highest-leverage decision for the eventual multi-tenant phase; retrofitting it later is expensive, adding it now is free.
- Audit log is append-only and separate from `tool_calls` — `tool_calls` records what happened technically; `audit_logs` records the governance decision trail (permission checks, policy decisions, approvals), which is the artifact customers and auditors actually care about.
- Agent identity is distinct from user identity — agents authenticate via signed API keys (`agent_identities`), not through the same OIDC flow as human dashboard users, because they are a different kind of principal with different lifecycle and revocation needs.

7. Non-Goals (Current Phase)

- Multi-tenancy is not yet enforced (schema supports it; Row-Level Security is not yet enabled) — see the separate Multi-Tenant SaaS Architecture document for that phase.
- Real tool integrations (CRM, payments) are mocked, not live.

- Billing, SSO/SAML, and multi-region deployment are out of scope until the funded phase — see the Bootstrap-to-Funded Stack document.

8. Related Documents

- docs/PROJECT_GUIDE.md — local setup, folder-by-folder guide, and coding conventions for new contributors.
- Multi-Tenant SaaS Architecture — the target architecture once selling to multiple organizations.
- Bootstrap-to-Funded Stack Plan — which pieces run on free infrastructure now and what they upgrade to.